

ORACLE

Update on an Abstraction for Verifiable Credentials and Zero Knowledge Proofs

Internet Identity Workshop, April 2025

Mark Moir, Harold Carr

Oracle Labs



Agenda

- 1 Background, overview of abstraction approach
- 2 Status: implementing and testing
- 3 Refining abstraction to accommodate different libraries
- 4 Above the abstraction
- 5 Wrapup

Agenda

- 1 **Background, overview of abstraction approach**
- 2 Status: implementing and testing
- 3 Refining abstraction to accommodate different libraries
- 4 Above the abstraction
- 5 Wrapup

About us



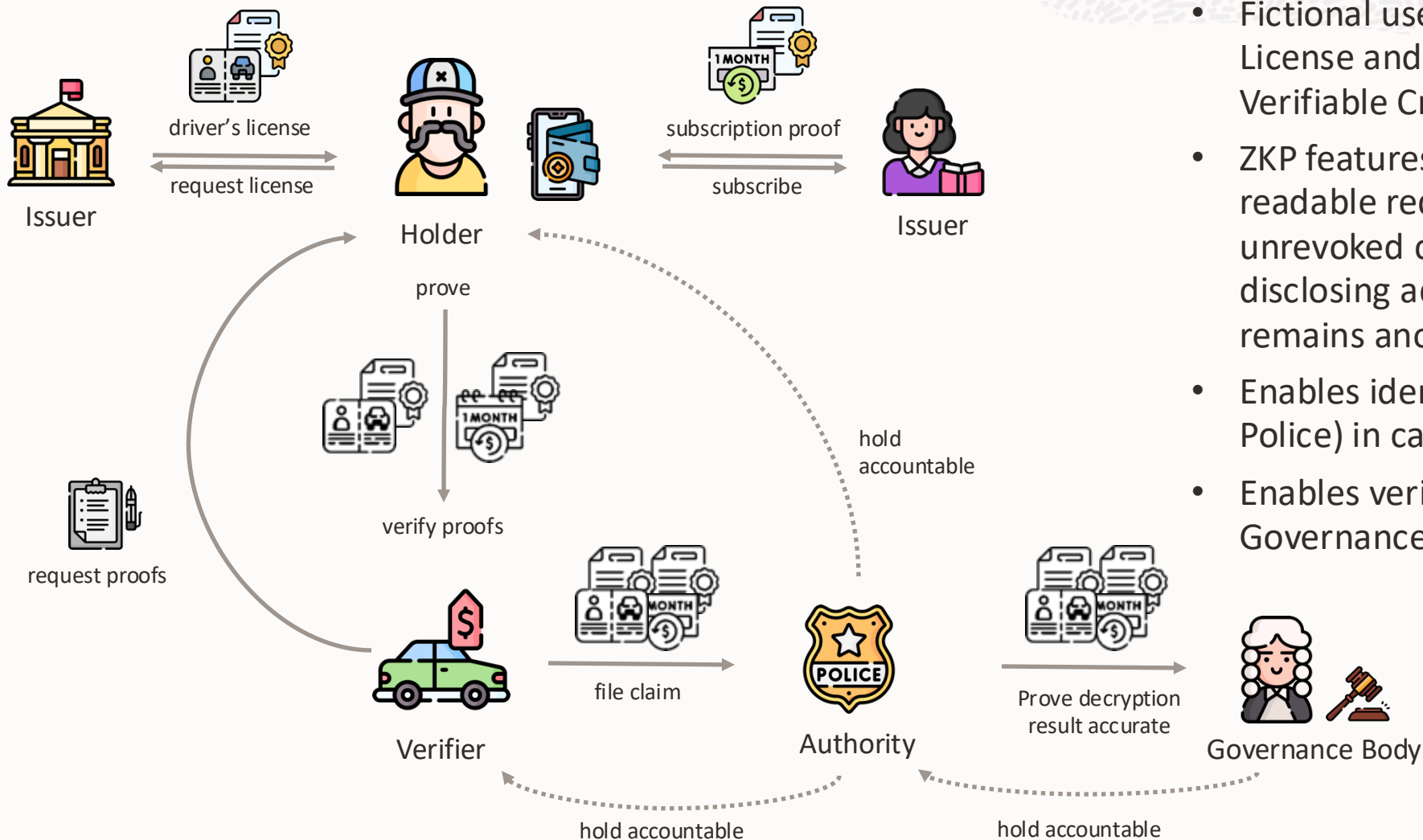
- Full-time
 - Harold Carr (<https://www.linkedin.com/in/haroldcarr>)
 - Blockchain fault tolerance and scalability, Infiniband transport, distributed systems, logic circuit simulation, ...
 - Mark Moir (<https://www.linkedin.com/in/markmoir>)
 - Blockchain fault tolerance and scalability, synchronization primitives, concurrent algorithms, formal verification, ...
- Former interns
 - Christoph Braun, Ph.D. student, Karlsruhe Institute of Technology
 - Contributed to use case demo and machinery to enable it (Summer, 2023)
 - Henry Blanchette, Ph.D. student, University of Maryland, College Park
 - Contributed to Haskell prototype and translating it to Rust (Summer, 2024)

“Elevator Pitch”: Using Verifiable Credentials and Zero Knowledge Proofs to balance privacy and accountability



- Collecting too much information creates liability, compliance and privacy issues
- Collecting too little precludes accountability
- Verifiable Credentials and Zero Knowledge Proofs can help to strike an effective balance
- But, many VC projects and standards don't enable such use of ZKPs and/or are tied too tightly to specific cryptography libraries, limiting choice and progress

Accountability in privacy-preserving authentication for services



- Fictional use case involving Driver's License and Monthly Subscription Verifiable Credentials
- ZKP features ensure hirer satisfies human-readable requirements (minimum age, unrevoked credentials, etc.), without disclosing additional information; hirer remains anonymous, untrackable
- Enables identification by Authority (e.g. Police) in case of accident/theft
- Enables verification of identification by Governance Body (e.g., Court)



Zero Knowledge Proof features



- To enhance privacy
 - Non-correlatable Proof of Knowledge of signature
 - Equality of attributes in multiple credentials
 - Blinded attributes (e.g., enables link secrets)
 - Selective Disclosure
 - Range proof (e.g., D.O.B. > 20 years ago)
 - Accumulator / set membership (e.g., to prove non-revocation)
- ... while ensuring accountability
 - Verifiable Encryption, Decryption, Verification of Decryption

Overview of our research

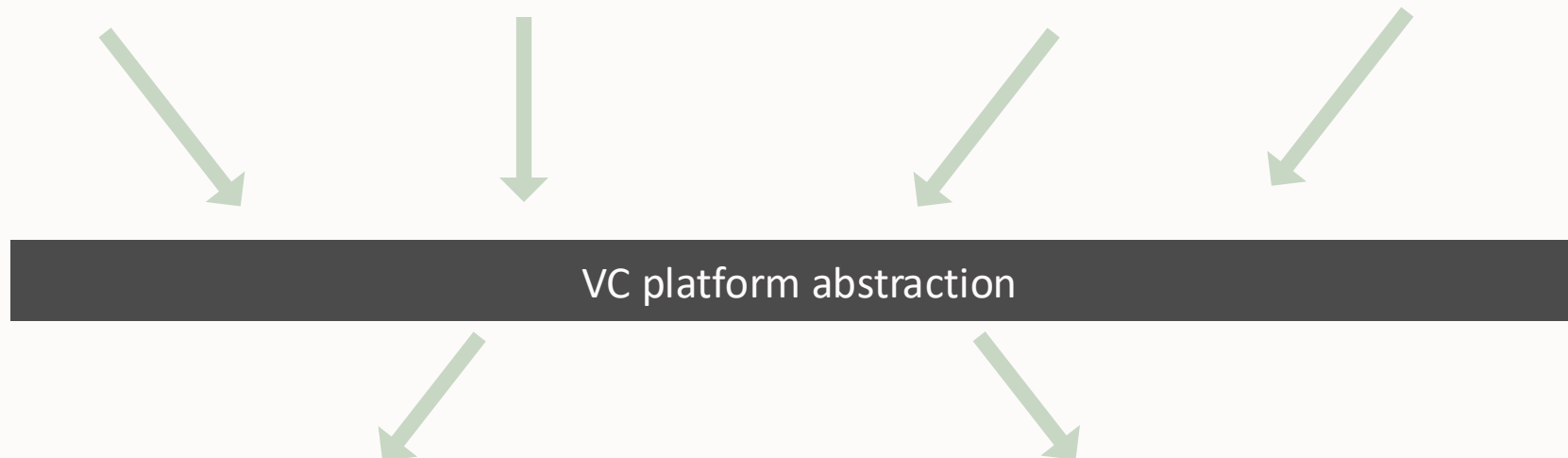


- Learning about these technologies, projects, standards efforts
- Demonstrating (internally) potential of technologies
- Developing an experimental abstraction to decouple VCs from underlying cryptography
- Implementing and testing our abstraction, instantiated with multiple cryptography libraries
- Seeking feedback and engagement internally and externally

An abstraction for decoupling credential formats from underlying cryptographic libraries

Credential formats

Presentation (Request) formats



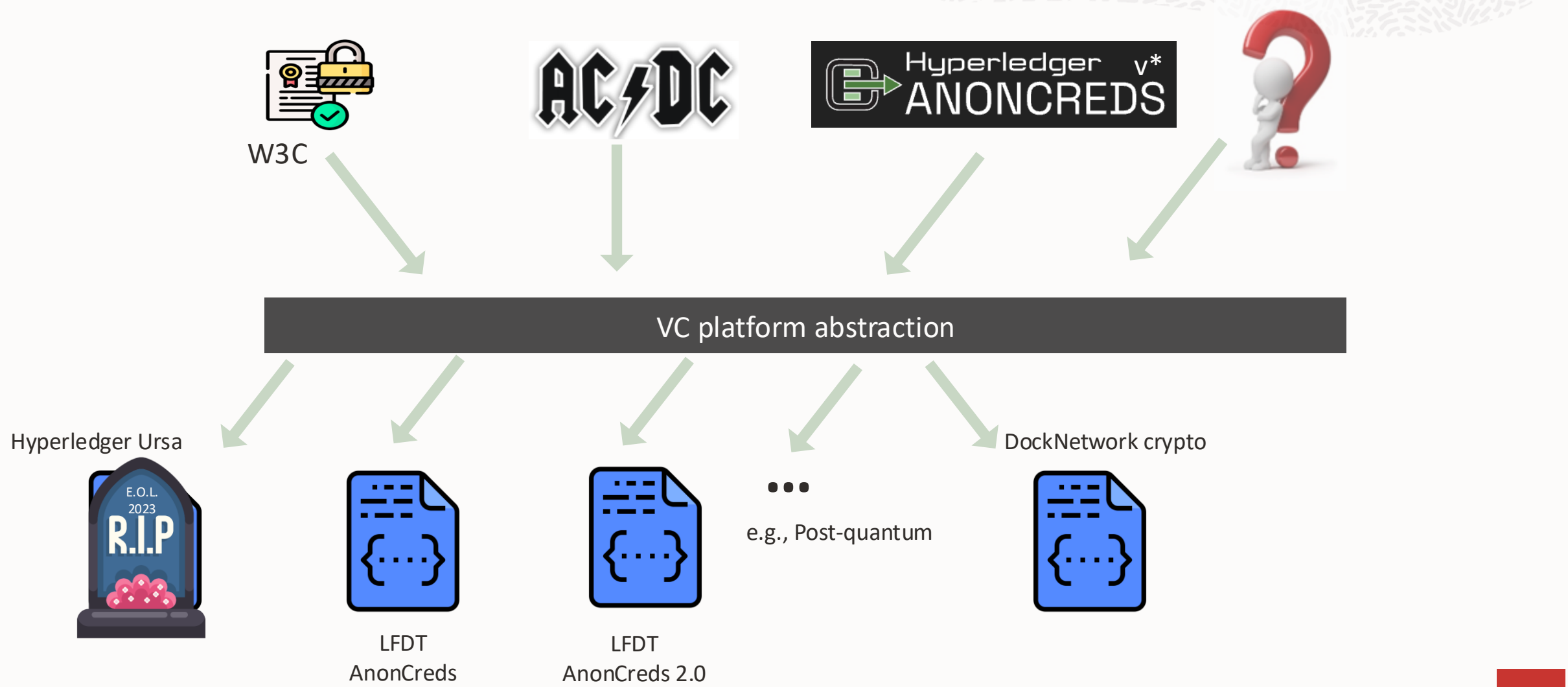
Cryptographic libraries



An abstraction for decoupling credential formats from underlying ZKP libraries



An abstraction for decoupling credential formats from underlying ZKP libraries



Motivation for abstraction



- Facilitate providing strong privacy and accountability features for any Credential and Presentation (Request) formats
- Facilitate adoption of new and/or improved cryptography libraries by any Credential and Presentation (Request) formats
- Enable evolution of Credential and/or Presentation (Request) formats and/or cryptography libraries, relatively independently of each other
- Facilitate experimentation with and evaluation of different credential formats, libraries, etc.
- Reduce risk for projects and products that aim to effectively balance privacy and accountability
- ...

Some potential benefits

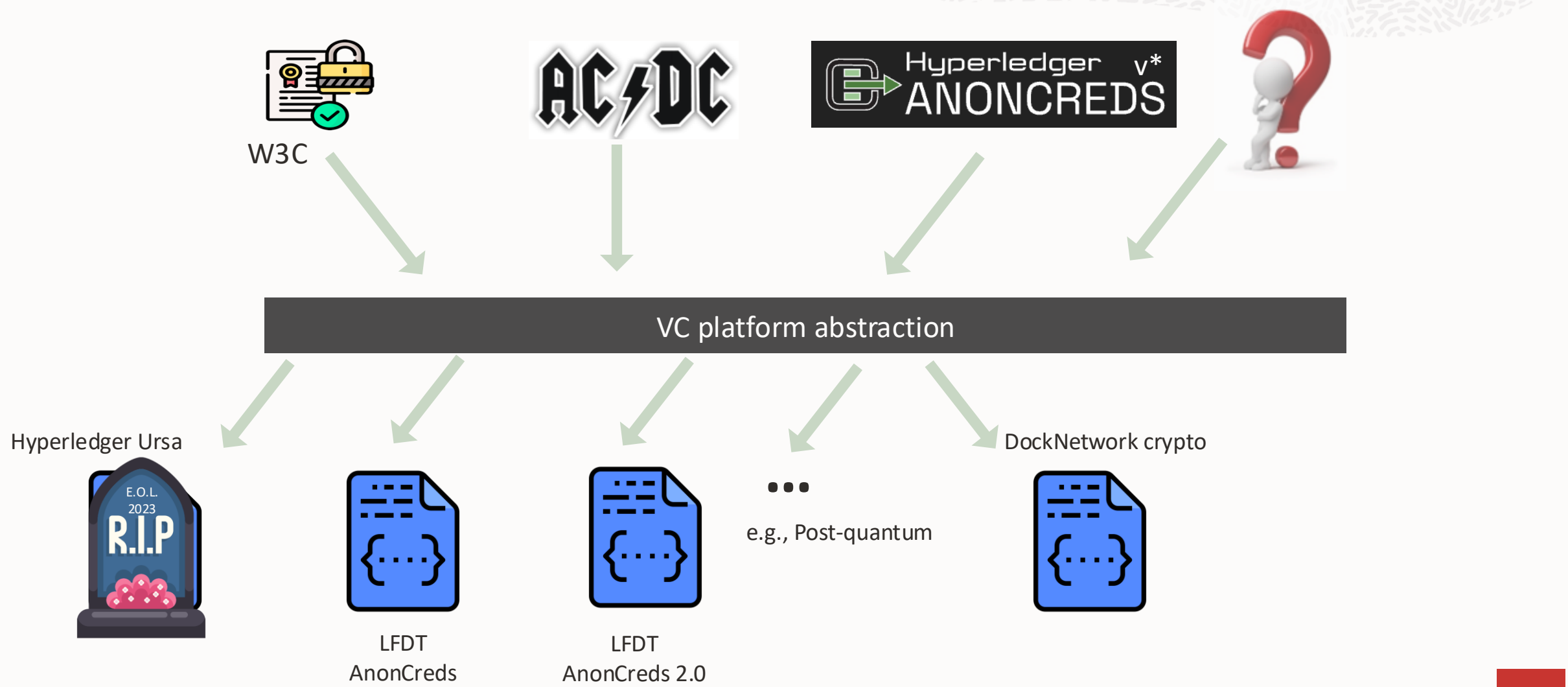


- Enable people (not just developers with expertise to use specialized cryptography libraries) to more easily express/understand/analyze/audit use case requirements in different ways
- Enable reports/summaries/warnings for use case requirements in various formats, levels of detail
 - Example: User requests to disclose a verifiably encrypted attribute: likely a mistake. Conceivably intentional, so we allow it, but generate a warning
 - Similarly for disclosing accumulator members (thus leaking correlatable info)
- Facilitate switching underlying cryptography libraries for various reasons:
 - Performance (comparison), features, stronger security guarantees (e.g., postnoquantum), more rigorous proofs, more favourable license, health of project, ...
- Could enable formal proofs of privacy properties about use cases, based on assumptions about guarantees made by underlying cryptography (stated formally, perhaps proved formally)
- Enable extensive and extendible test suites, independent of underlying library and implementation language

Agenda

- 1 Background, overview of abstraction approach
- 2 **Status: implementing and testing**
- 3 Refining abstraction to accommodate different libraries
- 4 Above the abstraction
- 5 Wrapup

An abstraction for decoupling credential formats from underlying ZKP libraries

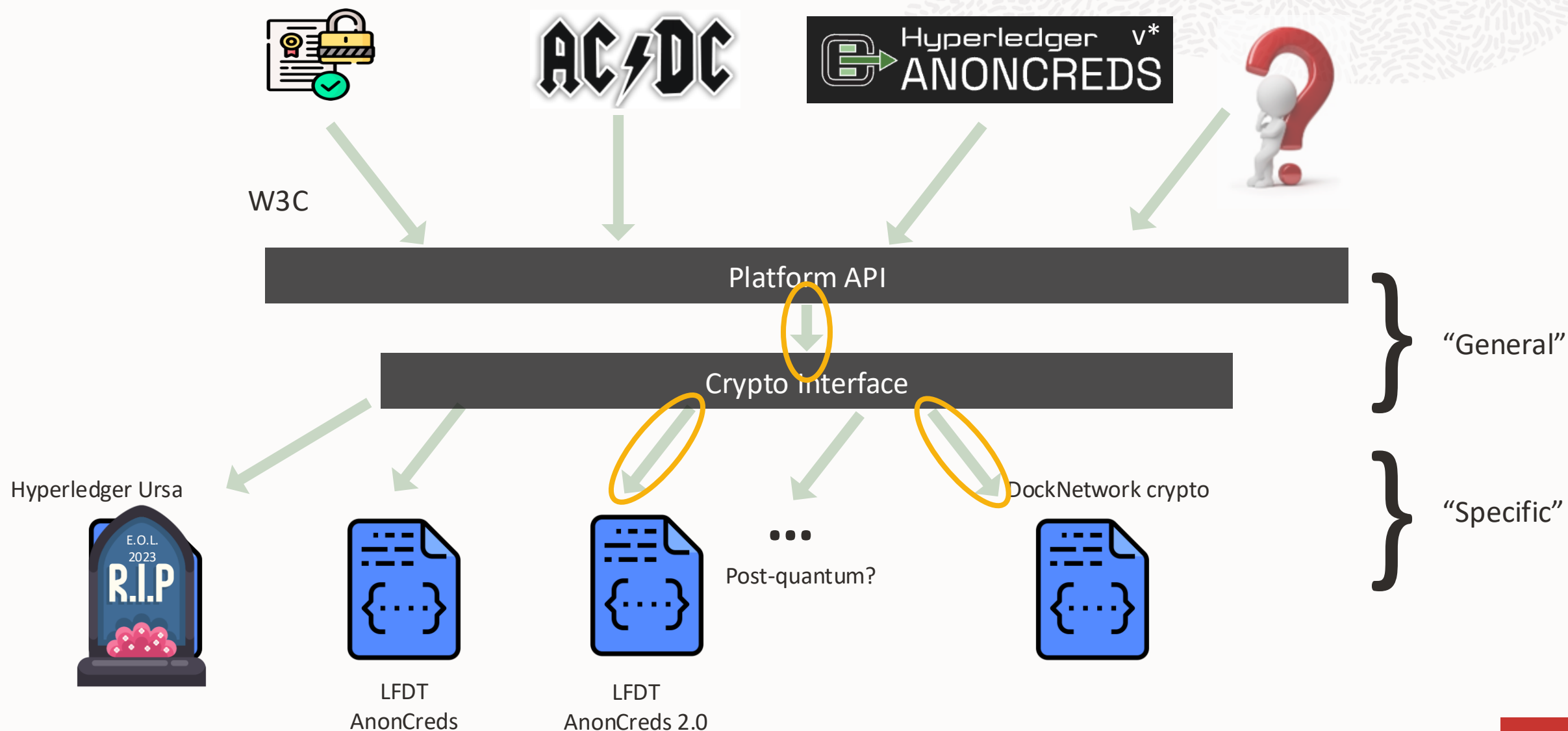


Actually, two related abstractions



- “Platform API”
 - Provides access to all ZKP-enabled features via higher-level abstraction, provides various validation, warnings, etc., while remaining independent of underlying library and Credential and Proof Request formats
- “Crypto Interface”
 - Defines minimal requirements for underlying ZKP library to implement

Separate minimal library requirements from higher-level functionality

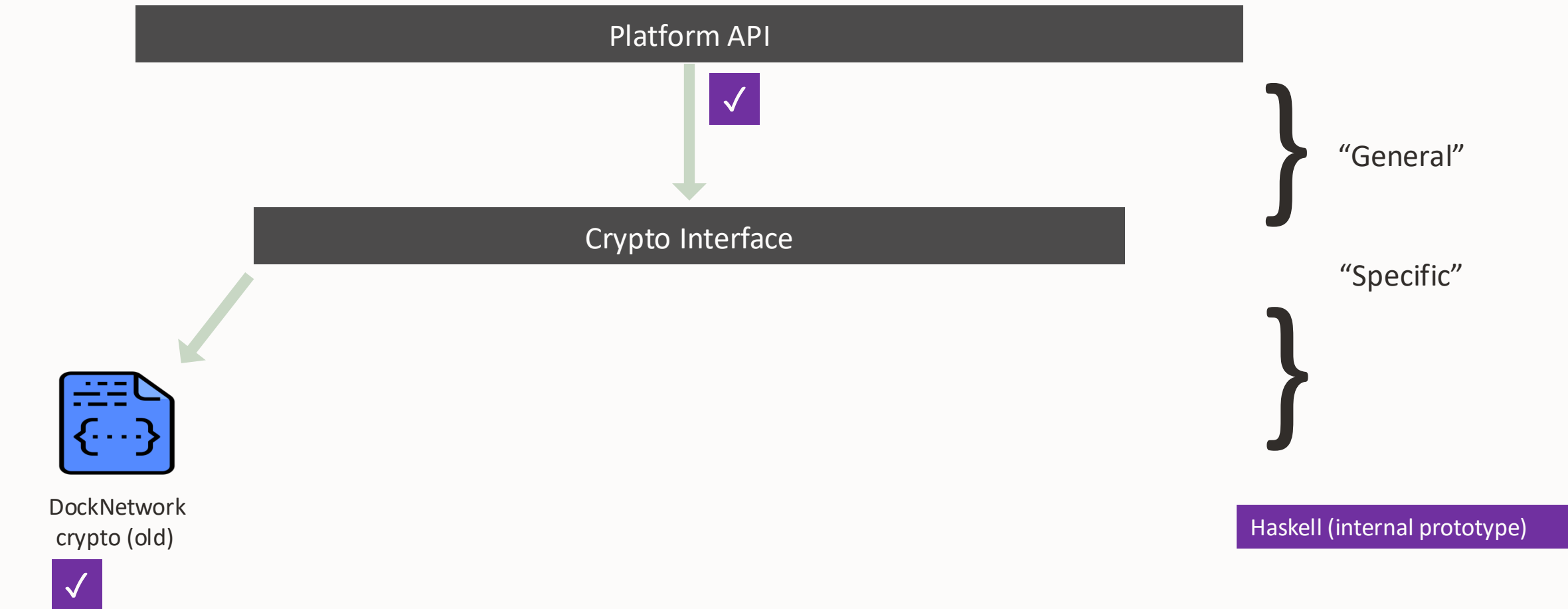


Benefits of “General” layer

- Reduce work for implementers of specific `CryptoInterface` by doing common work for them
- Uniformly, independent of credential format and ZKP library, ...
- Errors
 - You expressed a requirement for a non-existent attribute
 - You expressed a requirement for wrong type of attribute (e.g., Text attribute in range)
 - You’re not over 18, can’t produce a proof that you are
 - You’re asking to blind a non-blindable attribute
 - ...
- Warnings
 - Did you really mean to reveal an encryptable attribute?
 - Did you really mean to reveal an correlatable attribute, e.g., revocation ID?
 - ...

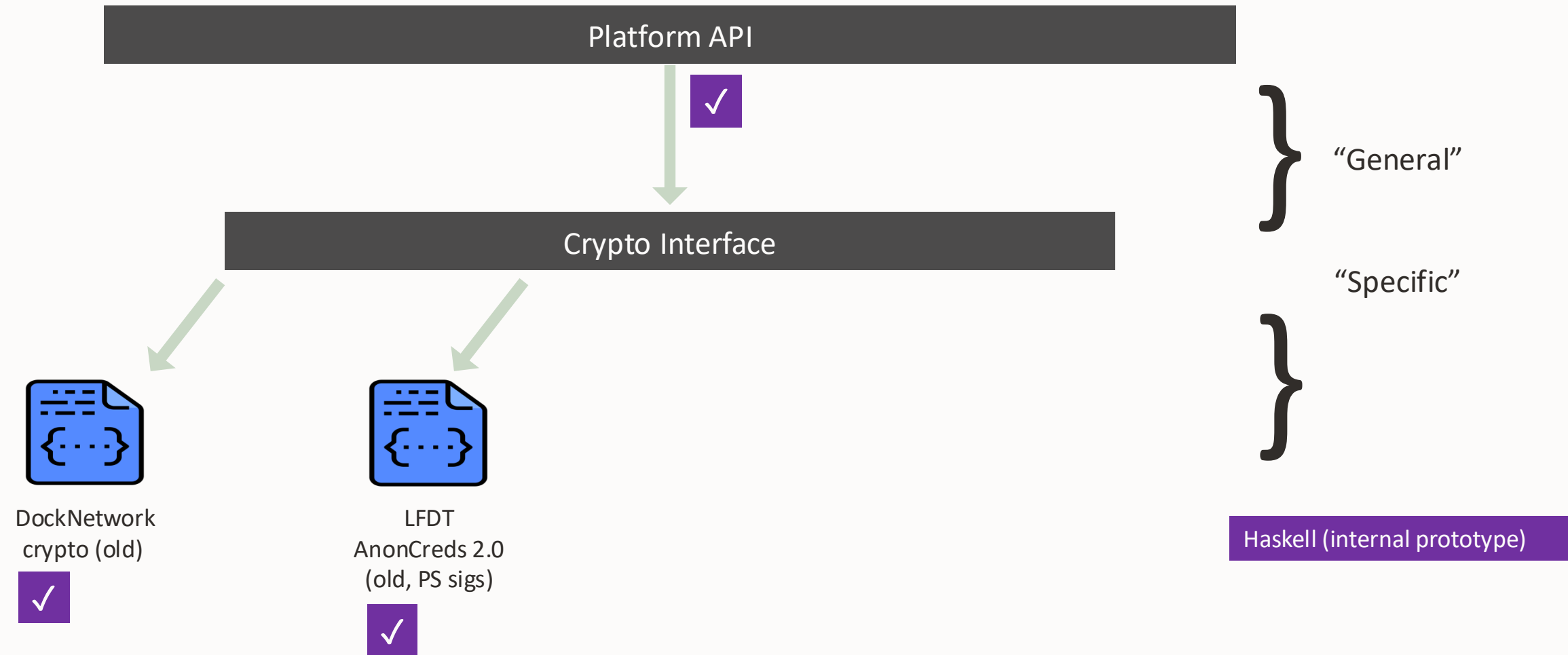
Implementing our abstraction over multiple ZKP libraries

Started with one ZKP library



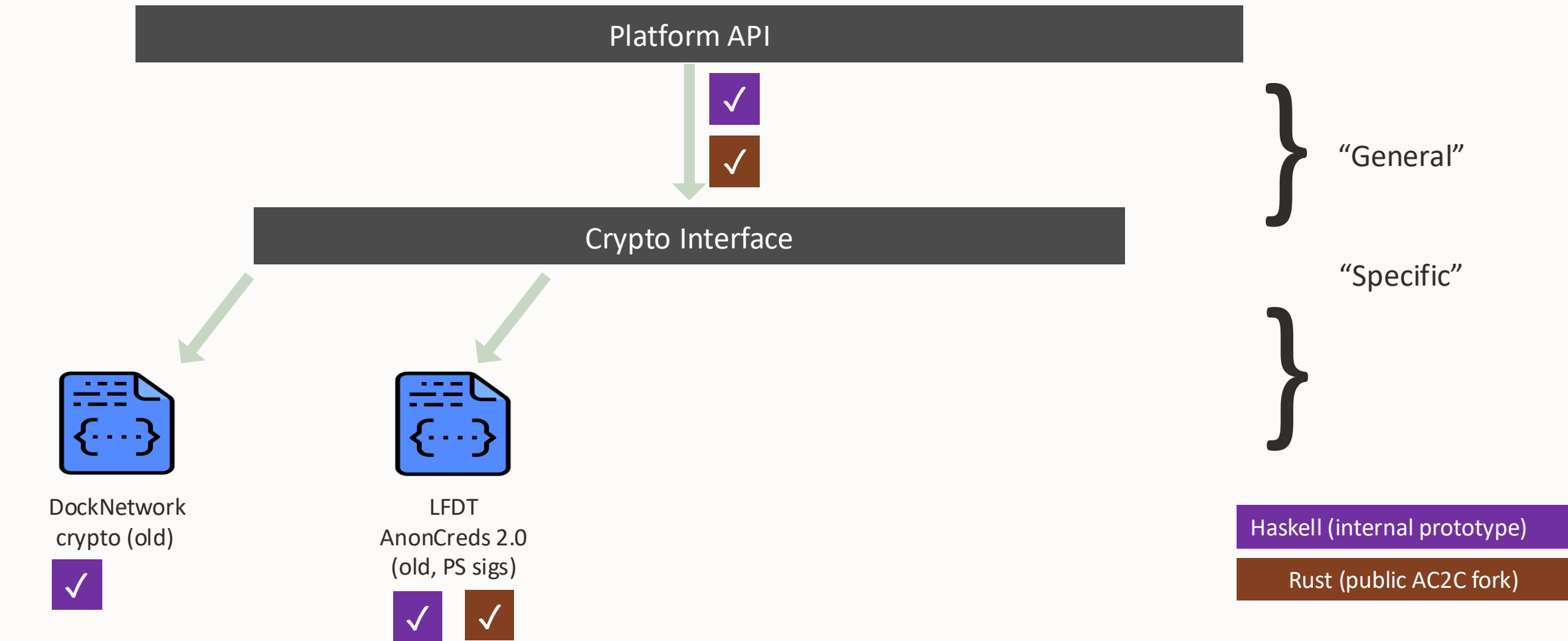
Implementing our abstraction over multiple ZKP libraries

Added another one



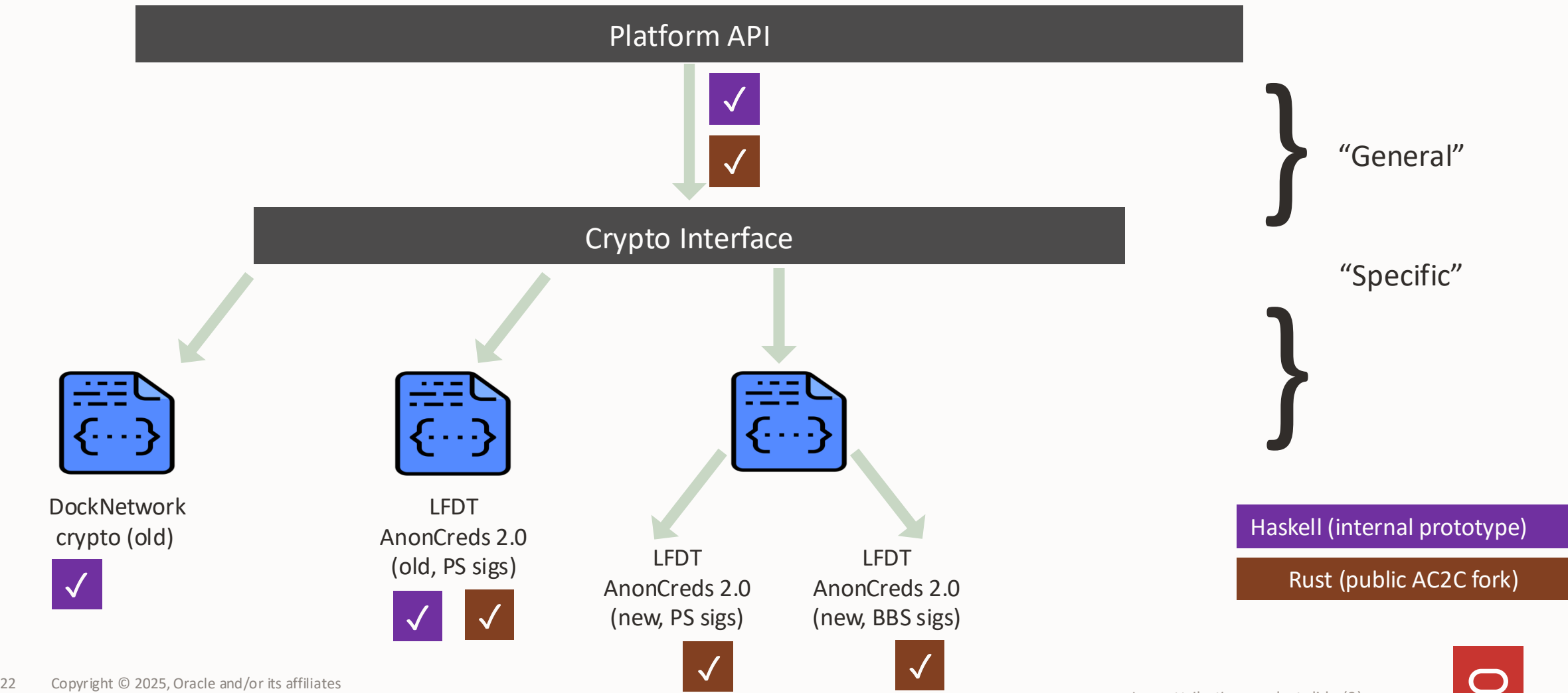
Implementing our abstraction over multiple ZKP libraries

Status as of IIW, October 2024



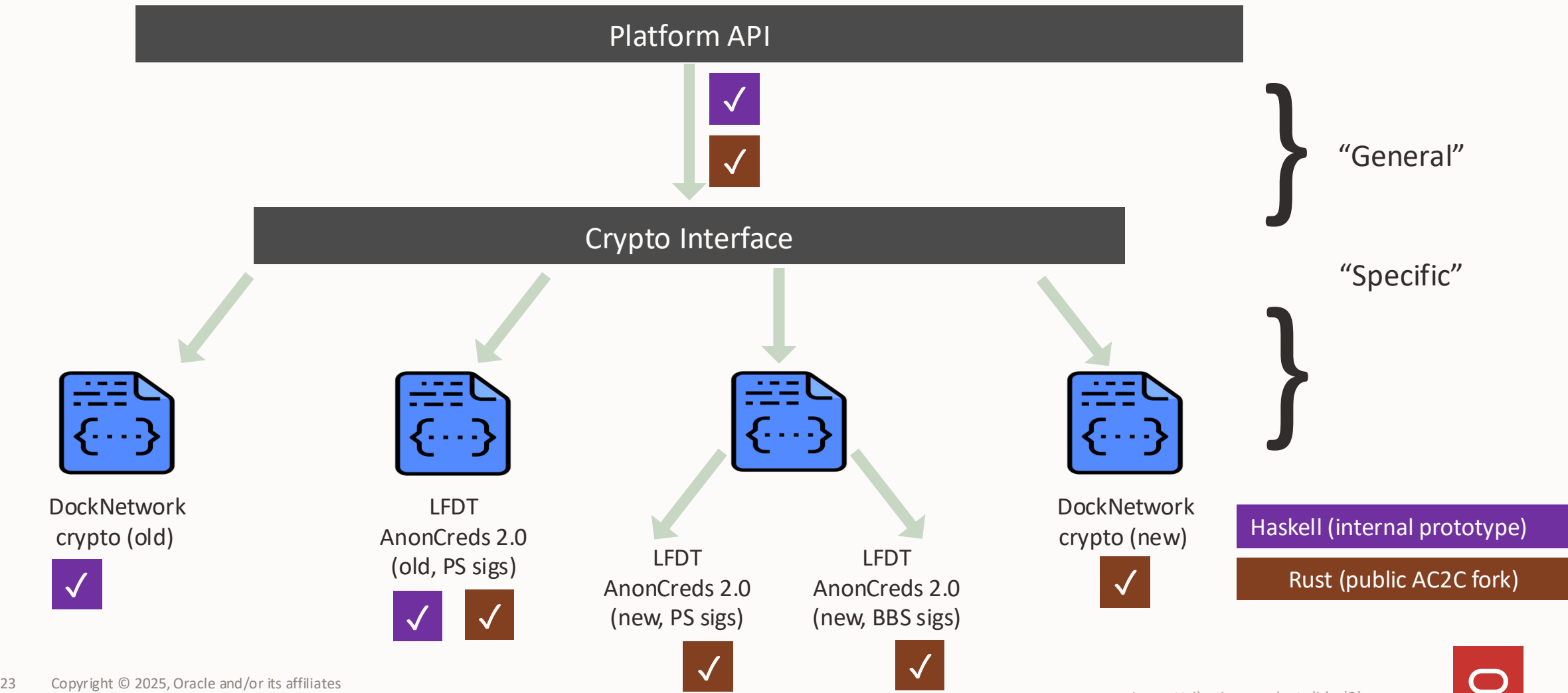
Implementing our abstraction over multiple ZKP libraries

Once AC2C supported "pluggable" signature schemes, we could immediately test with them



Implementing our abstraction over multiple ZKP libraries

Abstraction now also implemented over DockNetwork crypto in Rust



Test framework



- Test against any ZKP library that implements `CryptoInterface`
- Test suite currently includes 38 tests expressed in JSON, our test framework imports them and runs
- Additional tests added as JSON files, no programming required
 - <https://github.com/mark-moir/anoncreds-v2-rs/blob/vcp-main/src/vcp/README.md>
- Examples of additional tests beyond basic functionality that will help to evaluate any new/updated ZKP library that implements our abstraction:
 - Number and value of revealed and decrypted attributes are same as requested
 - Decryption verification succeeds for correct value, fails for “perturbed” value
 - Out-of-date revocation witnesses do not verify, updated ones do
 - Range proofs verify for boundary values, do not for out-of-range values
 - Equality proofs succeed for equal values, fail for different
 - Same value is provided for same attribute decrypted for multiple authorities

Feature and test coverage status (as of IIW, Oct 24)

	Haskell (internal)		Rust (open source)		JSON test coverage
	DNC	AC2C	DNC	AC2C	
PoK of Signature	✓	✓	☐	✓	✓
Selective disclosure	✓	✓	☐	✓	✓
Equality	✓	✓	☐	✓	✓
Range proof	✓	✓	☐	✓	✓
Set membership, including updates	✓	✓	☐	✓	✓
Verifiable Encryption	✓	✓	☐	✓	✓
Decryption	✓	NYS	☐	NYS	✓
Verify Decryption	✓	NYS	☐	NYS	✓

NYS = Not Yet Supported (by underlying cryptography library)



Feature and test coverage status

	Haskell (internal)		Rust (open source)		JSON test coverage
	DNC	AC2C (PS -> BBS)	DNC	AC2C (PS & BBS)	
PoK of Signature	✓	✓	✓	✓	✓
Selective disclosure	✓	✓	✓	✓	✓
Equality	✓	✓	✓	✓	✓
Range proof	✓	✓	✓	✓	✓
Set membership, including updates	✓	✓	✓	✓	✓
Verifiable Encryption	✓	✓	✓	✓	✓
Decryption	✓	TODO	✓	TODO	✓
Verify Decryption	✓	NYS	✓	NYS	✓
Blinded attributes (e.g., for link secret)	✓	✓	WIP	WIP	✓

NYS = Not Yet Supported (by underlying cryptography library)



Test framework overview



- `TestState` represents “state of the world”
 - All data for all roles (Issuers, Holders, Authorities)
 - Single, unnamed verifier
- `TestSteps` modify and/or check properties of current `TestState`, e.g.,
 - Create setup data (e.g., `IssuerData`, `AuthorityData`, ...), add requirements for Presentation by given Holder (PoK of sig, revealed attributes, equality, in range, in accumulator, encrypt for given Authority, decrypt by Authority, etc.)
 - Create and verify proof for given Holder, based on previously stated requirements
- Test steps include only human-friendly content, framework creates and manages Issuer/Authority/Accumulator setup params and keys, etc.

Example test scenario



- `CreateIssuer` (given label and schema)
- `CreateAccumulators` for Issuer (given its label)
- `SignCredential` (given labels for Issuer and Holder, and `DataValues`)
- `Reveal some attributes` (given Issuer and Holder labels, list of attribute indexes)
- `AccumulatorAddRemove` (given Issuer and Holder labels, attribute index, add value to accumulator)
- `InAccum` (given Issuer and Holder labels, attribute index, sequence number 1)
- `CreateAndVerifyProof`, expect both to succeed without warnings

Example test scenario – JSON (same test passes in Haskell and Rust)

```
{
  "descr": "example_single_issuer_and_credential_in_accum_no_update",
  "provenance": "DO NOT MODIFY MANUALLY: this test was generated programatically; ... <more info to enable us to identify original Haskell code>",
  "testseq": [
    {
      "contents": ["DMV",
        ["CTText", "CTInt", "CTText", "CTInt", "CTAccumulatorMember"]],
      "tag": "CreateIssuer"},
    {
      "contents": "DMV",
      "tag": "CreateAccumulators"},
    {
      "contents": ["DMV", "Holder1",
        [
          {
            "contents": "some metadata",
            "tag": "DVText"},
          {
            "contents": 37852,
            "tag": "DVInt"},
          {
            "contents": "Brown",
            "tag": "DVText"},
          {
            "contents": 180,
            "tag": "DVInt"},
          {
            "contents": "abcdef0123456789abcdef0123456789",
            "tag": "DVText"}
        ]],
      "tag": "SignCredential"},
    {
      "contents": ["DMV", 4,
        {
          "Holder1": {
            "contents": "abcdef0123456789abcdef0123456789",
            "tag": "DVText"}
        }],
      "tag": "AccumulatorAddRemove"},
    {
      "contents": ["Holder1", "DMV", [0, 3]],
      "tag": "Reveal"},
    {
      "contents": ["Holder1", "DMV", 4, 1],
      "tag": "InAccum"},
    {
      "contents": ["Holder1", "BothSucceed-Warnings"],
      "tag": "CreateAndVerifyProof"}
  ]
}
```

Example test scenario – Haskell (used to create JSON on previous slide)

```
example_single_issuer_and_credential_in_accum_no_update =
  TF.TestSequenceWithMetadata
    provenance
    "example_single_issuer_and_credential_in_accum_no_update"
    Nothing
    [ CreateIssuer "DMV"      [ CText, CInt, CText, CInt, CTAccumulatorMember ]
    , CreateAccumulators "DMV"
    , SignCredential "DMV" "Holder1" [ DVText "some metadata"
                                       , DVInt 37852
                                       , DVText "Brown"
                                       , DVInt 180
                                       , DVText "abcdef0123456789abcdef0123456789" ]
    , AccumulatorAddRemove "DMV" 4
      (fromList [("Holder1", DVText "abcdef0123456789abcdef0123456789")]) []
    , Reveal "Holder1" "DMV" [0, 3]
    , InAccum "Holder1" "DMV" 4 1
    , CreateAndVerifyProof "Holder1" BothSucceed-Warnings
    ]
```

TestSteps and their parameters (somewhat updated) 1/2

CreateIssuer	IssuerLabel [ClaimType] [CredAttrIndex]
SignCredential	IssuerLabel HolderLabel [DataValue]
CreateAccumulators	IssuerLabel
AccumulatorAddRemove	IssuerLabel CredAttrIndex (Map HolderLabel DataValue) [DataValue]
Reveal	HolderLabel IssuerLabel [CredAttrIndex]
InRange	HolderLabel IssuerLabel CredAttrIndex Word64 Word64
InAccum	HolderLabel IssuerLabel CredAttrIndex AccumulatorBatchSeqNo
Equality	HolderLabel IssuerLabel CredAttrIndex [(IssuerLabel, CredAttrIndex)]
CreateAndVerifyProof	HolderLabel CreateVerifyExpectation
UpdateAccumulatorWitness	HolderLabel IssuerLabel CredAttrIndex AccumulatorBatchSeq-
CreateAuthority	AuthorityLabel
EncryptFor	HolderLabel IssuerLabel CredAttrIndex AuthorityLabel
Decrypt	HolderLabel IssuerLabel CredAttrIndex AuthorityLabel
VerifyDecryption	HolderLabel

TestSteps and their parameters (somewhat updated) 2/2

- New, to support blinded attributes

```
CreateBlindSigningInfo      IssuerLabel [ (CredAttrIndex, DataValue) ] ProofMode
```

```
SignCredentialWithBlinding IssuerLabel HolderLabel [ (CredAttrIndex, DataValue) ] ProofMode
```

- The **SignCredentialWithBlinding** step:
 - retrieves Holder's previously-created `BlindInfoForSigner`
 - uses it to create a blinded signature
 - retrieves Holder's previously-created `InfoForUnblindingSig`
 - uses it to unblind blinded signature
 - stores the unblinded signature in the Holder's state.

Expectations, overrides, testing modes

- `CreateAndVerifyProof TestStep` takes a `CreateVerifyExpectation` (expected outcome of test). Test outcomes (so far):
 - `BothSucceedNoWarnings`
 - `CreateProofFails [Text]` (specify strings expected in error message)
 - `VerifyProofFails` (implies proof creation succeeds)
 - `CreateOrVerifyFails` (in some cases, some implementations refuse to create proofs, while others fail verification)
- Including `expected_to_fail` in a test name requires it to fail
- We can (per-cryptography-library) override (i.e., don't run) specific tests because:
 - `SLOW`
 - `Fail`: known to fail, e.g., because specific library has a known bug or limitation that causes tests to crash
 - `Skip`: don't want to run it for some other reason
- `ProofMode = Loose | Strict | TestBackend`

Testing both General and Specific layers



- Positive tests: succeed when credentials satisfy requirements
- Negative tests: fail when they don't, requirements are incorrect/inconsistent, etc.
- Want to confirm that expected errors are generated by “General” layer
- Also want to confirm that Prover cannot fool verifier simply by bypassing General-level checks
 - `TestBackend` mode skips General tests and passes arguments to ZKP library

Testing variants: out of range test (1/3)

```
{ "descr": "does_not_verify_out_of_range_negative_test_strict",  
  "provenance": "...",  
  "testseq": [...  
    { "contents": ["Holder1", "Strict",  
      { "contents":  
        ["...", "out of range", "attribute index"],  
        "tag": "CreateProofFails"} ]},  
    { "tag": "CreateAndVerifyProof"} ] }
```

- `Strict` mode, checks General-level error message generated as expected
- Passes for AC2C and DNC

Testing variants: out of range test (2/3)

```
{ "descr": "does_not_verify_out_of_range_negative_test_testbackend",  
  "provenance": "...",  
  "testseq": [...  
    { "contents": ["Holder1", "TestBackend",  
                  { "tag": "CreateOrVerifyFails" } ],  
    "tag": "CreateAndVerifyProof" } ] }
```

- **TestBackend mode**, bypasses General-level checking to test ZKP library behaviour
- **Expectation** is `CreateOrVerifyFails` because different libraries behave differently
- Passes for AC2C and DNC

Testing variants: out of range test (3/3)

```
{ "descr": "does_not_verify_..._test_testbackend_expected_to_fail",  
  "provenance": "...",  
  "testseq": [...  
    { "contents": ["Holder1", "TestBackend",  
      { "contents":  
        ["...", "out of range", "attribute index"],  
        "tag": "CreateProofFails" } ] },  
    { "tag": "CreateAndVerifyProof" } ] ] }
```

- Expects same user-friendly error from General-level checking, as before
- But TestBackend mode bypasses that checking
- `expected_to_fail` suffix in test name indicates it MUST fail
- Fails, enabling us to examine error messages from specific ZKP libraries:
 - AC2C*: `presentation_create: InvalidPresentationData`
 - DNC: `LegoGroth16Error(SynthesisError(Unsatisfiable))`

* Recent improvements in AC2C error messages not reflected, but still library-specific

Agenda

- 1 Background, overview of abstraction approach
- 2 Status: implementing and testing
- 3 **Refining abstraction to accommodate different libraries**
- 4 Above the abstraction
- 5 Wrapup

Expressing range proof requirements



- When we updated to a later version of DNC, some boundary tests that passed before started failing
- New DNC version interprets ranges as right-open, old was right-closed
- Also learned that AC2C and DNC support range proofs for different domains. Maximum value that can be proved to be within an expressible range is
 - AC2C: $2^{63} - 1$
 - DNC: $2^{64} - 2$
- Can't hope for all ZKP libraries to support precisely same semantics
- Extended our abstraction with `GetRangeProofMaxValue` operation to enable Issuers (in particular) to query to find out maximum value supportable in range proofs, enabling uniform expression of requirements

Blinded attributes



- AC2C requires to-be-blinded attributes to be “declared” when creating Issuer, and requires exactly that set of attributes to be blinded for each signature
- DNC more flexible
- Again, futile to require/hope all ZKP libraries support identical features
- Amusing side effect: although General-level checks ensure that “erroneous” use is rejected, one `TestBackend` test “fails to fail” with DNC, so we override it
- For now, we impose the requirement in our abstraction
- Could probably work around the requirement to “declare” blindable attributes, but we haven’t so far
- Question: is there any compelling use case that would make us revisit this decision?

Verifiable Encryption, Decryption, Verification of Decryption

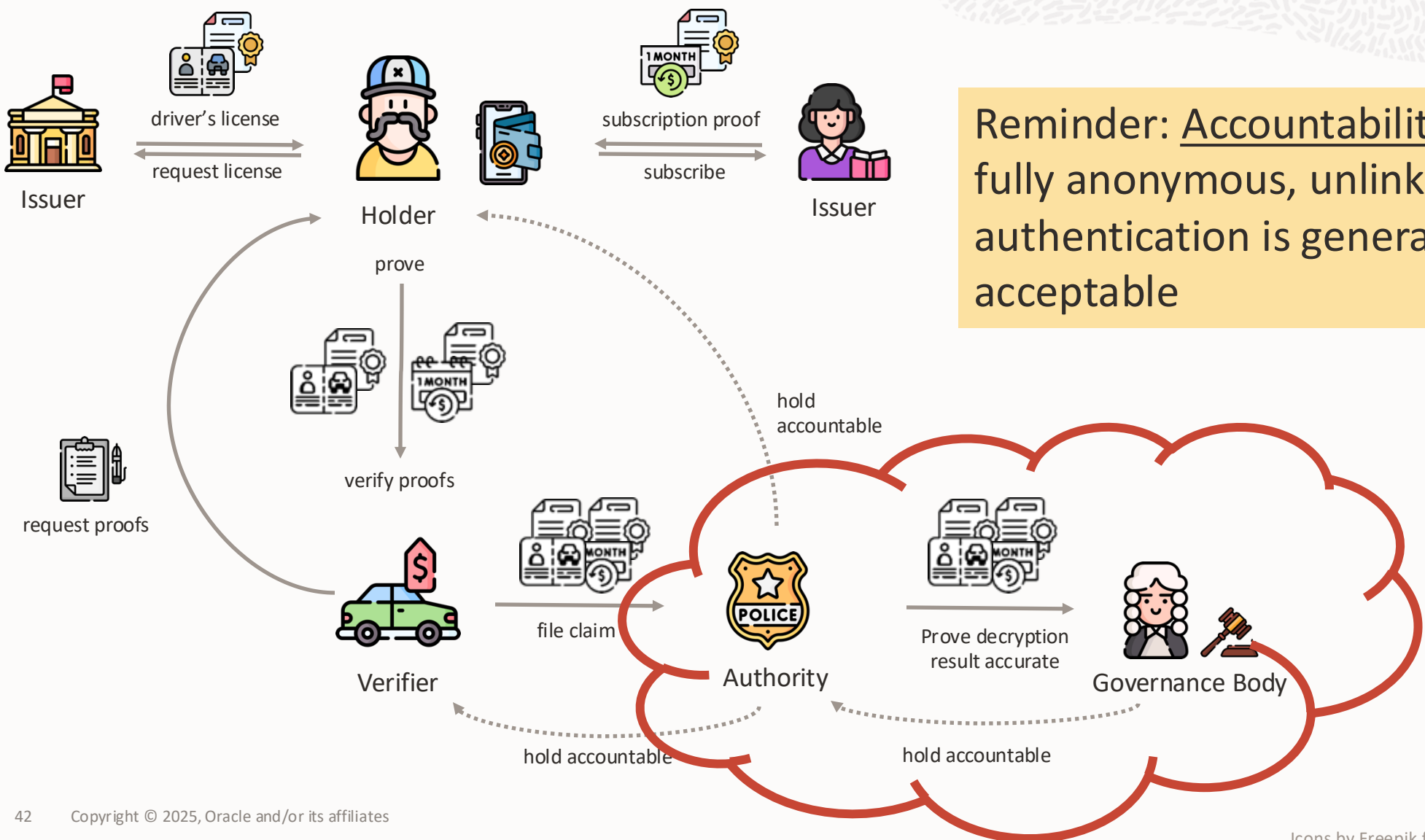
Implemented variants (summary from my understanding, TBC)

		Verifier		Authority		Governance Body		
		Size limit	Same as signed	Verify Encryption	Same as signed	Decrypt, Verify Encryption	Same as signed	Verify Encryption and Decryption
AC2C {	VerifiableEncryption (Auth. knows all values)	-	✓	✓	✓	✓	✓	-
	VerifiableEncryption (verifier decrypts for pseudonym)	-	✓	✓	N/A	N/A	N/A	N/A
	VerifiableEncryption	32B	✓	✓	✓	✓	✓	-
DNC	VerifiableEncryption -Decryption	-	-	✓	✓	✓	-	-
	SAVER	32B	✓	✓	✓	✓	✓	✓

- Different variants useful for different purpose/scenarios
- SAVER and VerifiableEncryption (scalar, limited to 32B) best for our purposes
 - 32B sufficient to encode information for accountability
 - Critical that Verifier can verify attribute signed in credential correctly encrypted for Authority
 - Verification of decryption by Governance Body “nice to have”, required for full implementation



Accountability in privacy-preserving authentication for services



Reminder: Accountability is key:
fully anonymous, unlinkable
authentication is generally not
acceptable



Summary of evolving abstraction



- Different (versions of) ZKP libraries have some differences in terms of at least
 - Features supported
 - Expression of requirements
 - Specific parameters of features
 - Required arguments
 - Performance
 - ...
- Highlights difficulty of changing from one to another, value of abstraction
- Our ongoing work has exposed some challenges and design decisions we didn't anticipate
- So far, abstraction approach has held up well

Agenda

- 1 Background, overview of abstraction approach
- 2 Status: implementing and testing
- 3 Refining abstraction to accommodate different libraries
- 4 **Above the abstraction**
- 5 Wrapup

Server and demo applications



- `http` server that serves our API via OpenAPI/Swagger
 - <https://github.com/mark-moir/anoncreds-v2-rs/blob/vcp-main/server/README.org>
- Internally, we have some small demo applications in several languages:
 - Java™, HTMX, TypeScript
- Java one is updated, works with latest API
 - <https://github.com/mark-moir/anoncreds-v2-rs/blob/vcp-main/server/example-client/java/README.org>

VCP server provides interactive documentation

Swagger

Supported by SMARTBEAR

..../openapi.json

Explore

vcp_server

0.1.0 OAS 3.0

..../openapi.json

default

POST

/vcp/createSignerData

Create the secret and public data used to sign and verify credentials.

POST

/vcp/createAccumulatorData

Create an accumulator and its associated secret/public data.

POST

/vcp/sign

Create a signature from the given values and SignerData.

POST

/vcp/createAccumulatorElement

Create an accumulator element from the given text.

POST

/vcp/accumulatorAddRemove

Add and/or remove elements from an accumulator.

POST

/vcp/updateAccumulatorWitness

Update an accumulator witness.

POST

/vcp/createMembershipProvingKey

Create accumulator membership proving key.

POST

/vcp/createRangeProofProvingKey

Create range proof proving key.

GET

/vcp/getRangeProofMaxValue

Get the maximum value supported in range proofs for the specific zkLib.

POST

/vcp/createAuthorityData

Create authority data. Used in verifiable encryption.

POST

/vcp/createProof

Create a proof with respect to proof requirements from a Verifier.

POST

/vcp/verifyProof

Verify a proof with respect to proof requirements.

POST

/vcp/verifyDecryption

Verify a decryption.

Filter

SEARCH

vcp_server

API Servers

Authentication

OPERATIONS

Add and/or remove elements from an accumulator.

Create an accumulator and its associated secret/public data.

Create an accumulator element from the given text.

Create authority data. Used in verifiable encryption.

Create accumulator membership proving key.

Create a proof with respect to proof requirements from a Verifier.

Create range proof proving key.

Create the secret and public data used to sign and verify credentials.

Get the maximum value supported in range proofs for the specific zkLib.

Create a signature from the given values and SignerData.

Update an accumulator witness.

Verify a decryption.

Verify a proof with respect to proof requirements.

Create a proof with respect to proof requirements from a Verifier.

POST /vcp/createProof

Returns WarningsAndDataForVerifier.

REQUEST

QUERY-STRING PARAMETERS

zkLib

string

Which ZKP library to use. AC2C_BBS, AC2C_PS or DNC (error if missing).

REQUEST BODY* application/json

EXAMPLE SCHEMA

OBJECT

Information used for creating a proof.

```
{
  "proofReqs": {
    "any-key": {
      "signerLabel": string
      "disclosed": integer
    }
    "inAccum": [
      {
        "index": integer
        "accumulatorPublicDataLabel": string
        "membershipProvingKeyLabel": string
        "accumulatorLabel": string
        "accumulatorSeqNumLabel": string
      }
    ]
    "notInAccum": [
      {
        "index": integer
        "label": string
      }
    ]
    "inRange": [
      {
        "index": integer
        "minLabel": string
        "maxLabel": string
        "rangeProvingKeyLabel": string
      }
    ]
  }
}
```

Proof requirements as specified by the Verifier.

Proof requirements for a specific credential.

A key into SharedParams to obtain the Signer's public data.

Specifies which attributes are to be selectively disclosed.

Constraints: Min 0

Specifies which attributes are to be proved PRESENT in a specified accumulator.

→ [Used to prove accumulator membership.]

Indicates which attribute is to be proved present in an accumulator.

Constraints: Min 0

A key into SharedParams to obtain an accumulator's public keys.

A key into SharedParams to obtain a MembershipProvingKey.

A key into SharedParams to obtain an Accumulator value.

A key into SharedParams to obtain a sequence number. A holder needs this to find the appropriate witness.

Specifies which attributes are to be proved ABSENT from a specified accumulator.

→ [An index-label pair.]

An index specifying an attribute.

Constraints: Min 0

A key into SharedParams.

Specifies which attributes are to be proved to be within a specified range.

→ [Information for range proof requirements.]

Specifies the index of the attribute to be proved to be within a specified range.

Constraints: Min 0

A key into SharedParams used to obtain the minimum value in the specified range.

A key into SharedParams used to obtain the maximum value in the specified range.

A key into SharedParams used to obtain the proving key to be used for the required range proof.



AnonCreds v1



- Planning to target AnonCreds v1 Credential and Presentation Request formats to our abstraction
- Initially, no change to formats
 - Enables substituting, e.g., BBS signature scheme, experimenting with unmodified applications, tests, use cases, etc.
- Beyond that, experiment with extending Presentation Request format to support additional features enabled by ZKP libraries, e.g., Verifiable Encryption, different revocation schemes, etc.
- Planning for this work motivated us to integrate Blinded Attributes, as AnonCreds v1 requires link secrets

Agenda

- 1 Background, overview of abstraction approach
- 2 Status: implementing and testing
- 3 Refining abstraction to accommodate different libraries
- 4 Above the abstraction
- 5 **Wrapup**

Contributions as byproducts of our work



- Additional/extended tests for ZKP libraries
 - To increase coverage and aid understanding how to use features
 - To demonstrate bugs we have identified and enable testing fixes
- Small Pull Requests with
 - Bug fixes
 - Improve usability/generality (e.g., ensure Prover and Verifier can generate their own Presentation Requests, so neither has to accept one generated by the other)
- Report issues
 - Via Github issues, or privately where appropriate
- Code reviews
- Feedback and input on feature requirements, suitability

Summary



- We are:
 - Learning about technologies, projects, and standards efforts around Verifiable Credentials and Zero Knowledge Proofs
 - Developing an abstraction to decouple VCs from underlying ZKP cryptography libraries
 - Implementing and testing our abstraction, instantiated with multiple ZKP libraries
 - Facilitating experimentation by others via a no-code test framework and an `http` server
- Recent progress includes:
 - Contributing to various improvements to open source ZKP libraries via identifying issues and submitting Pull Requests
 - Public implementation of our abstraction over multiple ZKP libraries, enabling no-code testing as well as a server serving our API for experimentation
 - <https://github.com/mark-moir/anoncreds-v2-rs/blob/vcp-main/src/vcp/README.md>
- More detail in [slides](#) for IIW39 (Internet Identity Workshop, October 2024)

Thank you

Mark Moir, Architect, Oracle Labs

mark.moir@oracle.com

www.linkedin.com/in/markmoir



Icon attributions



All icons by Freepik from flaticon.com, except:

1. Private message icons created by juicy_fish – Flaticon
2. Arrest icons created by shmai – Flaticon
3. Icon by ultimatearm
4. By Denmokin - Own work, Public Domain
5. Professional icons created by Uniconlabs - Flaticon